

CMCMIS · SAC · ISRO

PHASE 6 — SEALED

Job Requests + Job Cards · Slice 1

Locked Architectural Reference · No Source · Pure Doctrine

“Code can be re-typed. Doctrine, once forgotten, must be re-discovered the hard way.”

Field	Value
Document type	Phase Seal · Locked Architectural Reference
Project	CMCMIS_SIMPLIFIED (v1.0 FINAL · locked 16 May 2026)
Phase	Phase 6 — Job Requests + Job Cards · Slice 1
Companion doc	phase6codesHOWTOUSE.docx (build guide with source)
Author	Claude · Technical Documentation Engineer mode
Prepared for	Deep Sorathiya (DS)
Scope	Architecture · doctrine · decisions · patterns · diagrams
Out of scope	Source code · file-by-file listings (see companion doc)
Status	SEALED — additive amendments only
Version	1.0 · 2026-05-18

Document Control & Reading Guide

This is the sealed reference for Phase 6. Unlike the companion build guide, it contains no source code. Instead, it captures the architectural reasoning that produced the code, the principles that future phases will inherit, the trade-offs we accepted, and the patterns we chose. Read this document when you need to understand why something is the way it is. Open the companion document when you need to know exactly which characters to type.

This document is for...

New engineers joining the project who need to absorb the doctrine before touching code.
Code reviewers who want to defend or challenge a design decision.
Architects scoping Slice 2 and beyond who need to know what choices are still open.
Auditors verifying that controls (BR-JR-06, T&C double-lock, audit pairing) are real and traceable.
Future-you, six months from now, who will not remember why the repo aliases canonical names.

This document is NOT for...

Typing in code — open the companion doc for that.
Quick how-tos — open the companion doc.
Step-by-step migrations — open the companion doc.
Smoke-test curl commands — open the companion doc.
Marketing or executive summaries — this is engineering documentation.

Reading order

Twelve parts. You can read them in order on a long evening, or jump straight to a part when you need it. The first three parts set up the vocabulary and the worldview — skip them at your peril if you are new to the project.

Part	Title	Pages (approx.)	When to read
I	Mission Briefing	5	Day one. Always.
II	The Sealed Context (Phases 0–5)	6	Day one or whenever a previous phase is referenced.
III	The Build Doctrine	10	Before designing any module.
IV	The Database Story	10	Before writing any SQL or migration.
V	Backend Module Architecture	12	When implementing or reviewing backend code.
VI	The Job Request State Machine	8	Before any change to status mutation logic.
VII	Frontend Architecture & Data Flow	10	Before adding pages or components.
VIII	The Three Screens	10	Before designing new UI flows.

Part	Title	Pages (approx.)	When to read
IX	Verification Methodology	8	Before signing off on a slice.
X	Operational Readiness	6	Before any deploy to staging.
XI	Troubleshooting Doctrine	8	When something is broken.
XII	Appendices & Decision Log	10	Reference. Search-only.

i Visual vocabulary used throughout this document

DEFINITION cards (ivory) — establish a term so the rest of the section can use it without ambiguity.

PRINCIPLE cards (sage) — non-negotiable design laws that all of Phase 6 (and future phases) obey.

PATTERN cards (periwinkle)— named recurring shapes you will see in many places.

DECISION cards (sky) — explicit choices with the alternatives we rejected and why.

HOW-TO-THINK cards (amber)— mental models you should adopt before reading the implementation.

TRADE-OFF cards (violet) — places where we knowingly accepted a cost for a benefit.

WHAT-WE-DID cards (mint) — single-line summaries closing each major section.

PITFALL cards (rose) — failure modes you must actively avoid.

MODULE cards (slate) — name, role, files, and dependencies of one backend or frontend module.

Glossary — Terms Used in This Document

The vocabulary below is used everywhere. When in doubt, return to this page.

Term	Meaning
Canonical name	The service-layer name for a column or field. Decoupled from the real DB column name so the SQL layer can absorb schema drift without forcing changes upstream.
Choke-point	A single place in the codebase through which all instances of a certain operation must pass. Phase 6 has one choke-point per state machine (job-request transitions and job-card transitions, each in its own state machine module).
Defence in depth	The practice of enforcing the same rule at multiple layers — client UI, client schema, network, server schema, server logic, database constraint — so a single layer breach does not produce a violation.
Doctrine	A binding rule of practice, written down so it survives turnover. Not a guideline, not a suggestion.
Idempotent	Safe to apply repeatedly. Every Phase 6 migration is idempotent by design — running it twice has the same effect as running it once.
Repo (repository) layer	The only layer of the backend that knows real DB column names. Aliases them to canonical names on SELECT and accepts canonical names on INSERT.
Row-level scope	Server-side filtering that restricts a list query to rows the caller is allowed to see, applied transparently via middleware rather than per-endpoint code.
Sealed phase	A phase whose artifacts are frozen. Additive amendments allowed (new columns, new permissions, new routes); destructive changes forbidden (no rename, no drop).
Slice	A vertical, end-to-end shipping unit of a phase. Slice 1 = list + create draft + submit. Slice 2 = approve/reject and the auto-Job-Card creation hook.
State machine	A pure-function module that owns all legal transitions for one entity. Throws on illegal transitions. No I/O.
Submitter snapshot	The four denormalised columns (name, employee id, designation, email) that the server fills in from the authenticated user at draft creation. Snapshot, not foreign key — so audits remain accurate even if the user is renamed later.
T&C double-lock	The Terms-and-Conditions gate enforced on both the client (button disabled until 6/6 boxes checked) and the server (literal-true zod check on tnc_accepted). Either alone is insufficient.
BR-JR-06	Business rule: the Submitted-By section is set server-side from req.user. The client cannot supply or override it.
BR-VIS-01	Visibility rule: a Normal User sees only the job requests they themselves submitted.
BR-VIS-02	Visibility rule: a Lab Engineer sees their own assignments plus the unassigned job-card queue.

Term	Meaning
BR-AUD-01	Audit rule: every state change writes exactly one audit row inside the same transaction. No state change without an audit row; no audit row without a state change.
Phase 3	The database phase. Sealed. ~70 tables. ALTER ADD allowed; DROP / RENAME forbidden.
Phase 4	The authentication phase. Provides JWT cookies, authenticate middleware, authorize/authorizeAny middleware, role+permission seed.
Phase 5	The equipment module. Provides the layered-module pattern used as the template for Phase 6.
Acceptance criterion (A1–A12)	A numbered, testable statement of correctness. Each has a smoke test mapped to it. All twelve must pass before the slice ships.

Part I — Mission Briefing

Phase 6 ships the first vertical slice of the job-flow system: the operational heart of CMCMIS. Three screens, three new database concepts, and one new way for the system to track work from request to assignment. Everything else in the system already exists — auth, equipment, divisions, roles. We are adding the verb that turns CMCMIS from a registry into a workflow.

DEFINITION Phase 6 Slice 1

A focused, shippable increment that delivers three screens: (1) a paginated list of job requests with filters, (2) a multi-section creation form gated by terms and conditions, and (3) a paginated list of job cards joined to their parent requests.

It does NOT include detail pages, approve/reject mutations, the auto-create-job-card hook, observation entry, or PDF generation. Those land in Slice 2 and Slice 3.

It DOES include the durable foundation that all subsequent slices will sit on: state machines, audit pairing, code generation, row-level scope, and the layered module pattern.

Why "slice", not "feature"

A slice is vertical: it cuts through every architectural layer from database to UI. A feature is horizontal: it spans many entities at one layer. We deliberately ship slices because each one proves the full pipeline works end-to-end before we commit to the next, and because a broken slice is a clear, contained problem.

PRINCIPLE Three is the right number of screens for Slice 1

Two screens would leave the system unbalanced — you could create requests but never see the downstream consequence (job cards).

Four screens would force premature design of the detail view and the approve/reject UI, which both depend on architectural choices (form state, optimistic update strategy) that we want to make later when we have data.

Three is the smallest number that proves: (a) read works, (b) write works, (c) the join from request → card exists in production form.

The three screens

Screen	Route	Primary Verb	Visible to
Job Requests · List	/job-requests	READ (own or all, depending on role)	All five roles
Job Requests · New	/job-requests/new	CREATE (draft) + SUBMIT (state change)	Normal · Lab Eng. · LIC · SA

Screen	Route	Primary Verb	Visible to
Job Cards · List	/job-cards	READ (filtered by row-level scope)	View-Only · Lab Eng. · LIC · SA

What the five roles see

Role	Job Requests List	New Request	Job Cards List
Normal User	OWN rows only	Yes	No
View-Only	ALL rows (read-only)	No	ALL rows
Lab Engineer	ALL rows	Yes	OWN assignments + ASSIGNED queue
Lab In-Charge	ALL rows (plus future approve UI)	Yes	ALL rows
Super Admin	ALL rows	Yes	ALL rows

HOW TO THINK Roles are not just labels — they shape the SQL

When you read "Normal User", do not picture a checkbox in the UI. Picture a WHERE clause added to every list query against job_requests, scoped to submitted_by_user_id = caller.id.

When you read "Lab Engineer + Job Cards", do not picture a permission flag. Picture an OR clause that widens the WHERE to include the unassigned queue (status = "ASSIGNED").

This is the rowLevelScope middleware in action. Roles produce SQL fragments, not just UI gates. Phase 4 (authenticate + authorize) handles the UI gates. Phase 6 adds the SQL fragments.

What we are NOT shipping in Slice 1

! Out of scope — defer to a later slice

Job Request detail view (read-only single-record page).

Approve / Reject buttons for Lab In-Charge.

Re-assignment of a job card to a different engineer.

The auto-create-job-card hook that fires on approve (it is hinted at in the state machine table but not wired).

PDF generation for job cards or calibration certificates.

Dashboard widgets that summarise job-request counts.

Inquiry tab integration.

Bulk export to CSV / Excel from the list pages.

Why these are deferred (not dropped)

Each deferred item depends on a design decision we want to make with real production data behind us. Detail views need empirical answers about what fields people actually scroll to. The approve/reject UI must consider edge cases like "approved with reservations" that we do not yet have evidence for. PDF generation involves licensing, fonts, and layout choices that benefit from concrete user requests rather than speculative templates.

TRADE-OFF Shipping less now to ship more durably later

Cost of slicing: a Normal User cannot yet click into a request and read all its details — they can only see the list row.

Benefit of slicing: the production database accumulates real rows before we design the detail view, so its layout reflects real reading patterns instead of guesses.

Net judgement: the cost is recoverable in days. The benefit compounds for the lifetime of the system.

WHAT WE DID Mission briefing closure

Defined Slice 1 as three screens that prove the request-to-card pipeline end-to-end.

Mapped the five roles onto concrete row-level filters, not just UI gates.

Made the out-of-scope list explicit so no one accidentally builds a deferred feature.

Established the doctrine that we ship vertical slices, not horizontal features.

Part II — The Sealed Context (Phases 0 through 5)

Phase 6 does not exist in isolation. It rests on five sealed phases. Understanding what each provides — and what each forbids us to change — is more important than understanding any single Phase 6 file.

Phase status snapshot

Phase	Name	Sealed?	What it locks in
0	Discovery	YES	23 clarifying questions resolved. The product brief is frozen.
1	Domain	YES	Personas, business rules, functional requirements. The vocabulary of the system.
2	Architecture	YES	Twenty technology decisions and six stack additions. The shape of every module.
3	Database	YES	~70 tables, ERD, seed scripts. ALTER ADD allowed; DROP / RENAME forbidden.
4	Authentication	YES	JWT cookies, role+permission matrix, authenticate/authorize middleware.
5	Equipment	YES	The layered-module pattern. Phase 6 copies its shape file-for-file.
6	Jobs (this slice)	IN PROGRESS	List + draft + submit. State machine. Audit pairing.

PRINCIPLE Sealed means additive, not immutable

You may ADD a column to a Phase 3 table — every Phase 6 migration does this.

You may ADD a permission code to the Phase 4 seed — Phase 6 adds eight.

You may ADD a route to the Phase 5 server — Phase 6 adds three.

You may NEVER rename, drop, repurpose, or change the type of an existing artifact. The sealed contract is "previous code still runs unchanged" — and additive amendments satisfy that contract.

What each previous phase contributes to Phase 6

MODULE Phase 0 — Discovery

Contributes: the FINAL-DESC document, which is the spec we map every endpoint and screen back to.

Reference points: §6.4 (roles), §6.5 (permission codes), §8.1 (state machine), §8.3 (choke-point pattern), §9.6 (row-level scope).

Phase 6 dependency: we read it, we cite it, we never modify it.

MODULE Phase 1 — Domain

Contributes: business rules (BR-JR-06, BR-VIS-01, BR-VIS-02, BR-AUD-01) and the role taxonomy.
Phase 6 dependency: every guard in code traces back to a business rule by ID. No business rule means no enforcement code.

MODULE Phase 2 — Architecture

Contributes: the layered-module pattern (validators → state-machine → repo → service → controller → routes), the choice of zod for validation, the choice of mysql2/promise for raw SQL, the choice of pino for logging.

Phase 6 dependency: we cannot pick a different validation library or skip a layer. The pattern is binding.

MODULE Phase 3 — Database

Contributes: tables job_requests, job_cards, equipment, users, divisions, audit_log, plus the migration discipline.

Phase 6 dependency: we discover the real schema with read-only introspection (Step 0 of the build), then add only columns and indexes our service layer needs (Step 1).

What we are forbidden to do: drop a column, rename a column, change a foreign-key target, alter an existing ENUM in place.

MODULE Phase 4 — Authentication

Contributes: the authenticate middleware (verifies JWT cookie, populates req.user), the authorize / authorizeAny middleware (checks permission strings), the cookie-handling utility, the refresh-token flow, and the role_permissions seed table.

Phase 6 dependency: every route file uses these middleware in the same order: authenticate → authorize → rowLevelScope → validate → controller.

What changed in Phase 4 to make Phase 6 possible: the role_permissions seed was extended with eight new codes (the job_request:* and job_card:* family).

MODULE Phase 5 — Equipment

Contributes: the canonical example of the layered-module pattern, copied file-for-file by Phase 6.

Phase 6 dependency: when you look at jobRequests.validators.js, you should recognise it as "the same shape as equipment.validators.js, with different fields". When you look at jobRequests.repo.js, you should recognise it as "the same shape as equipment.repo.js, with different SQL".

This sameness is intentional. New engineers learn one pattern and apply it to every module they read.

What Phase 6 contributes back to the system**WHAT WE DID Phase 6 brings the system to life**

A working request-to-card pipeline that proves CMCMIS is a workflow system, not just a registry.

A reusable state-machine module that future phases can copy for any other stateful entity (purchase orders, observation reports, calibration certificates).

A reusable code-generation utility that produces human-readable identifiers without contention.

A reusable audit-pairing pattern that ensures every state change leaves a permanent trail.

A reusable row-level-scope middleware that pushes role logic out of every endpoint and into one shared place.

The seal — what we promise about Phase 6**★ The Phase 6 seal**

Once Slice 3 closes, the entire jobs module is sealed.

Future slices and phases may add columns, add states, add permissions, add screens.

Future slices and phases may NEVER rewrite the state machine, remove the audit pairing, bypass the choke-point, or change the canonical-name mapping.

If you find yourself wanting to do any of those, the right answer is almost always: introduce a new state machine for the new behaviour, not modify the existing one.

Part III — The Build Doctrine

Doctrine is the set of laws every line of Phase 6 obeys. Phases 7, 8, 9 will inherit it. Future engineers will need to defend or amend it, never ignore it. This part is intentionally non-implementation-specific — it would apply just as well if we rewrote the system in Go or Python.

“Layers exist so that changes stay local.”

Doctrine 1 — The Bottom-Up Layering Law

PRINCIPLE A backend module is built in six layers and built bottom-up

Layer 1 · Validators — the contract. What shapes of data are legal.

Layer 2 · State Machine — the verbs. Which transitions exist and who may invoke them.

Layer 3 · Repo — the SQL. The only place that knows real DB column names.

Layer 4 · Service — the business logic. Where rules combine into use cases.

Layer 5 · Controller — the HTTP shim. Maps requests to service calls and back to JSON.

Layer 6 · Routes — the wiring. Express paths bound to middleware chains and controllers.

HOW TO THINK Why bottom-up instead of routes-first

Routes-first forces you to mock every layer beneath to compile. Mocks hide bugs.

Bottom-up means every file you save is testable in isolation the moment it lands. You catch errors at the layer where they originate.

Bottom-up also matches the dependency arrow: each layer imports only layers below it, never above. Build order = import order.

Top-down feels productive (you "see" routes early) but pays the bill later when a small validator change cascades through stubs.

The layered backend, visualised

Layer 6 · ROUTES imports → controller	(Express paths + middleware order)
Layer 5 · CONTROLLER imports → service	(req → service args; result → JSON)
Layer 4 · SERVICE imports → repo, state machine, utils	(business logic; transactions)
Layer 3 · REPO imports → db pool	(raw SQL; canonical-name aliasing)

Layer 2 · STATE MACHINE (pure function; transition table)
imports → nothing

Layer 1 · VALIDATORS (zod schemas; .strict() always)
imports → zod

↑↑↑ imports flow only upward, never downward ↑↑↑

PITFALL Layer-skipping is the most common architectural smell

A controller that writes raw SQL has skipped the repo and the service layer.

A route that calls the state machine directly has skipped the service layer.

A repo that calls an authorize() helper has reached upward into the controller domain.

If you find yourself doing any of these, refactor before merging. Hidden coupling compounds.

Doctrine 2 — The Canonical-Name Doctrine

PRINCIPLE Only the repo layer knows real DB column names

Every layer above the repo refers to columns by their canonical name (the name documented in SCHEMA_PHASE6.md).

The repo SELECT clause aliases real names to canonical names on the way out.

The repo INSERT/UPDATE binding maps canonical names to real names on the way in.

Result: if a real column is renamed (or has always had a surprising name), exactly one file changes.

DEFINITION SCHEMA_PHASE6.md

A hand-authored markdown file living at backend/db/discovery/SCHEMA_PHASE6.md.

Each row maps: canonical name (used in code) → real DB column name (used by SQL) → type → notes.

It is the single source of truth for the binding. Authored once in Step 0 of the build, updated whenever the real schema drifts, read by every engineer who needs to understand why a field has the name it does.

Why this matters

Without canonical naming

Every layer hard-codes real DB names.
A column rename forces edits to validators, services, controllers, and the frontend.
Schema drift is a multi-day refactor.
New engineers must memorise legacy column names from day one.

With canonical naming

Only the repo knows the real name.
A column rename is a one-line repo change.
Schema drift is a half-hour task.
New engineers learn canonical names that read like English.

Doctrine 3 — The Single Choke-Point Pattern

PRINCIPLE Every state change for an entity passes through exactly one function

For job requests, that function is transition() inside jobRequests.stateMachine.js.

For job cards, the equivalent will be jobCards.stateMachine.js (added in Slice 2).

No service, controller, or test bypasses the choke-point to update status directly.

No raw UPDATE status SQL appears anywhere outside the corresponding state-machine module.

PATTERN The Choke-Point Pattern

Anatomy: a pure function takes (currentState, action, actor, context) and returns either a newState or throws a structured error.

It does not touch the database. It does not log. It does not have side effects.
It is trivially testable: every transition can be exercised with a single function call and an assertion.
Used everywhere there is a finite-state machine — job requests, job cards, future purchase orders, future calibration certificates.

HOW TO THINK Why pure-function state machines win

They make the legal transition table the single source of truth, not scattered if/else branches in services.

They make permission checks explicit per-transition (who can submit? who can approve?) rather than implicit per-endpoint.

They make audit trails easy: every successful transition produces (fromStatus, toStatus, action, actor) which is exactly the shape of a status-history row.

They make illegal-transition errors precise: clients see "cannot reject a DRAFT request" instead of generic 400s.

Doctrine 4 — Idempotent, Additive Migrations

PRINCIPLE Migrations are append-only and re-runnable

Every ALTER is wrapped in an INFORMATION_SCHEMA existence check. If the column or index already exists, the migration is a no-op.

Every CREATE TABLE uses IF NOT EXISTS.

No DROP statement, no RENAME statement, no destructive ALTER appears in any Phase 6 migration file.

Result: any migration can be applied to any environment any number of times, in any order, with the same final state.

TRADE-OFF Idempotency costs verbosity

A simple "ALTER TABLE ... ADD COLUMN ..." becomes a CALL to a helper procedure that checks first.

Each migration file is 30-50% longer than its naive equivalent.

The benefit: deploys never fail because someone re-ran a script, and rollback-by-redeploy is a one-line operation.

Net: the verbosity is paid once per file; the deploy-safety pays every time we ship.

Doctrine 5 — Defence in Depth

PRINCIPLE Every guard is enforced at multiple layers

BR-JR-06: enforced at the form (no inputs for submitter fields), at the FE schema (no such keys), and at the BE schema (.strict() rejects unknown keys).

T&C double-lock: enforced at the FE (submit button disabled until 6/6 checked) AND at the BE (zod literal-true check).

Row-level visibility: enforced at the middleware (sets req.scope) AND at the repo (WHERE clause uses req.scope).

Audit pairing: enforced by service-layer convention AND verified by smoke-test #6 (count rows in audit_log).

HOW TO THINK Why one layer is never enough

A clever client can bypass the FE — DevTools enables a disabled button in five seconds.

A buggy middleware can fail to set req.scope — without a repo-layer check, the user sees other people's rows.

Defence in depth means each layer is a complete defence on its own. Two complete defences make the system survive a single failure.

Doctrine 6 — The Audit-Pairing Rule

PRINCIPLE One state change = one audit row in the same transaction

The service layer opens a database transaction.

Within that transaction it (a) writes the state-changed row via the repo, and (b) writes the audit_log row via the audit helper.

Either both succeed and the transaction commits, or both fail and the transaction rolls back.

A successful state change without an audit row is impossible by construction.

An audit row pointing at a rolled-back state change is impossible by construction.

PITFALL Audit logging outside the transaction is the silent killer

It is tempting to write the audit row after the transaction commits — it feels safe ("commit succeeded, so log the success").

It is wrong. If the audit insert fails (full disk, connection drop), the state has changed but the trail is gone.

Worse: if a developer later wraps the audit insert in a try-catch to "make it robust", silent failure becomes the default behaviour.

The audit helper accepts a connection parameter precisely to make in-transaction logging the easiest path.

Doctrine 7 — Code Generation Without Contention

PRINCIPLE Human-readable identifiers, atomic increments, no row locks

JR codes (JR-2026-NNNN) and JC codes (JC-2026-NNNN) are generated by the codeGenerator utility.

The technique: INSERT ... ON DUPLICATE KEY UPDATE next_seq = LAST_INSERT_ID(next_seq + 1) against a code_counters table.

No SELECT FOR UPDATE. No row-level lock held across an HTTP round-trip. No risk of long-lock deadlocks under load.

Sequences are scoped per (entity, year) — JR-2026 restarts at 0001 every January, JC-2026 too.

TRADE-OFF Sequential codes leak volume

Anyone with a request code can estimate how many requests the system has processed this year (JR-2026-0847 → about 850 so far).

For an internal MIS this is acceptable. For an external customer-facing system it would not be.

If we ever externalise the API we will switch to a UUID with a vanity prefix (JR-2026-A7F3K9). The codeGenerator interface is stable enough to absorb that change at one file.

Doctrine 8 — The Thin Controller Rule

PRINCIPLE Controllers contain HTTP, not business logic

A controller reads req.body, req.query, req.params, and req.user; it calls a service function with normalised arguments; it sets a status code and returns res.json.

If a controller has a conditional that depends on a business rule, that conditional belongs in the service layer.

If a controller catches an error to map it to a status code, that mapping is centralised — controllers just rethrow into next().

HOW TO THINK Why thinness is a feature

A thin controller is trivially testable as integration glue — supertest is all you need.

A thin controller is reusable across transports — the same service function feeds a GraphQL resolver or a CLI command with no rewrite.

A thin controller is replaceable — when you swap Express for Fastify, you rewrite the controller layer only.

Summary of the eight doctrines

#	Doctrine	Enforced by
1	Bottom-up layering	Module folder structure. Import-graph linter rule.
2	Canonical naming	SCHEMA_PHASE6.md hand-authored mapping; aliases in repo SELECTs.
3	Single choke-point	One state-machine file per entity. No raw UPDATE status outside it.
4	Idempotent, additive migrations	INFORMATION_SCHEMA guards; CREATE IF NOT EXISTS; no DROP statements.
5	Defence in depth	Multi-layer guards (FE schema, BE schema, middleware, repo WHERE).
6	Audit pairing	Service-layer convention; same connection passed to writeAudit().
7	Atomic code generation	INSERT-on-duplicate-key idiom; per-(entity, year) scoping.
8	Thin controllers	No business logic in controllers; rethrow via next(); centralised error middleware.

WHAT WE DID Build doctrine closure

Captured the eight non-negotiable laws that govern every line of Phase 6 code.

Made it impossible to write code that violates one of them without leaving a visible smell (raw UPDATE status, layer-skipping import, audit outside transaction).

Provided trade-off transparency so the next engineer can defend each doctrine in review.

Part IV — The Database Story

Phase 3 sealed the database with roughly seventy tables. Phase 6 needed to add information without breaking that seal. This part is the story of how we did it — read-only discovery first, additive migrations second, performance indexing third, never destructive at any step.

The four-step database protocol

Step	Action	Files involved	Effect
1	Read-only introspection	0001_phase6_introspect.sql	Output captured to a text file. Zero schema change.
2	Author canonical map	SCHEMA_PHASE6.md (hand-authored)	Documents canonical-to-real bindings. Zero schema change.
3	Apply additive column migrations	0010_phase6_columns.sql	Adds missing columns guarded by existence checks.
4	Apply index and history migrations	0011_phase6_indexes.sql + 0012_status_history.sql	Adds five BTREE + one FULLTEXT + history tables.

Step 1 — Read-only introspection

DEFINITION Read-only introspection

A SQL script consisting entirely of SHOW CREATE TABLE statements plus simple COUNT(*) queries.

Run against the dev database. Output captured to a file. Never run against production.

Purpose: discover the real shape of every table Phase 6 will touch — column names, types, defaults, foreign keys, indexes.

HOW TO THINK Why introspection comes before any other work

The Phase 3 sealing document records what tables exist but engineers drift columns over time. Reality may have moved.

Writing code against assumed columns and discovering at runtime that they have different names is the slowest possible debugging path.

Twenty minutes of introspection saves four hours of confused stack traces.

Introspection is also the first time you see the size of the real data (COUNT(*)) — which informs whether your indexes are even necessary.

Step 2 — The canonical-to-real map

After introspection produces text output, the engineer hand-writes SCHEMA_PHASE6.md mapping the canonical name (used in code) to the real DB column name (used in SQL). Every canonical field gets exactly one row. Missing columns are flagged so Step 3 can add them.

PATTERN The Canonical Map Pattern

A markdown table with five columns: canonical, real, type, nullable, notes.

Lives at backend/db/discovery/SCHEMA_PHASE6.md alongside the introspection output.

Updated by hand whenever real schema drifts; never auto-generated (auto-generation hides intent).

Cited by every repo file in a top-of-file comment so reviewers know where the binding came from.

Step 3 — Additive column migrations

When introspection reveals that the real DB lacks a column the canonical model expects, the migration adds it. Every ADD is guarded by an INFORMATION_SCHEMA check so re-running the migration is a no-op. No column is ever dropped or renamed.

Columns Phase 6 needed to add

Table	Column	Why
job_requests	request_code	JR-2026-NNNN identity. Used in URLs, lists, audit logs.
job_requests	priority	ENUM(LOW, MEDIUM, HIGH). Drives sort and color in the list.
job_requests	accessories	JSON array. Multi-row alternative was rejected because accessories never need to be searched independently.
job_requests	tnc_accepted_at, tnc_version	Evidence of T&C acceptance. tnc_version isolates legal-text revisions.
job_requests	submitted_at	Wall-clock when DRAFT became SUBMITTED. Distinct from created_at (which is when the draft was created).
job_requests	division_code	Denormalised from divisions table for fast list rendering without a JOIN.
job_requests	submitted_by_name / emp_id / designation / email	BR-JR-06 snapshot. Survives user renames.
job_cards	card_code	JC-2026-NNNN identity.
job_cards	equipment_name, assigned_engineer_name	Denormalised for fast list rendering.

DECISION JSON array for accessories instead of an accessories child table · *JSON vs child table*

Decision: store accessories as a JSON array on job_requests, not as rows in a child table.

Alternative considered: a job_request_accessories table with (request_id, type, name, serial_no) tuples.

Why JSON won: accessories are never queried independently — never "find all requests that include a Fluke 8845 accessory". They are read and written together as a group.

Cost: schema-less denormalisation means application-layer validation is the only contract. We accept this in exchange for read-side simplicity and zero join overhead.

Reversibility: if accessories ever need indexed search, the JSON can be projected into a new child table without changing the public API.

DECISION Submitter snapshot columns instead of foreign key only · *Snapshot vs join*

Decision: store submitted_by_name, submitted_by_emp_id, submitted_by_designation, submitted_by_email as denormalised columns at draft creation.

Alternative considered: rely on the foreign key submitted_by_user_id and JOIN users for the rendered name.

Why snapshot won: a user can be renamed, transferred, or deactivated. The audit-grade record of "who submitted this request on 12 May 2026" must show the name as it was on that day.

Cost: a tiny storage overhead per row and an obligation to keep the snapshot in sync with the FK at write time.

Reversibility: the snapshot is additive — the FK is still there, queries can still join, the snapshot just survives joins.

Step 4 — Indexes and history

Two index families plus two history tables. The indexes are designed to make the most-common queries index-only scans. The history tables provide a permanent log of state transitions for forensic queries.

The five BTREE indexes on job_requests

Index name	Columns	Query it covers
idx_jr_list_default	(status, created_at DESC, id)	Default unfiltered list (sort=-created_at).
idx_jr_owner	(submitted_by_user_id, created_at DESC)	Normal-User list filtered by row-level scope.
idx_jr_division	(division_id, created_at DESC)	Filtered list when a division is selected.
idx_jr_priority_status	(priority, status, created_at DESC)	Sort by priority (HIGH first).
idx_jr_code	(request_code)	Lookup or search by request code.

HOW TO THINK What "covering" means and why it matters

A covering index contains every column the query needs. The engine never reads the heap; it returns rows from the index alone.

For a list endpoint that serves p50 < 50 ms, covering is the difference between 5 ms and 50 ms at 10,000 rows.

You verify covering by running EXPLAIN and looking for "Using index" in the Extra column. If you see "Using where" alone, the index is not fully covering.

The FULLTEXT index on job_requests

DEFINITION ft_jr_search

A FULLTEXT index on (request_code, equipment_name, serial_no, submitted_by_name).

Backs the search box on the Job Requests list. A user types "fluke" and matches against any of the four columns at near-zero cost.

Falls back to LIKE for very short queries (< 4 chars) where FULLTEXT does not help. The fallback is implemented in the repo with an OR clause.

TRADE-OFF FULLTEXT vs LIKE for the search box

FULLTEXT is fast (sub-millisecond at 100k rows) but has a 4-character minimum and ignores common short words.

LIKE %term% is universal but linear in row count — fine at 1k rows, painful at 1M.

Our solution: use FULLTEXT for terms of 4+ characters, fall back to LIKE on request_code only for shorter terms. Best of both at small implementation cost.

The two history tables

Table	Purpose	Row written when
job_request_status_history	Permanent log of every state transition on job_requests.	A service-layer transition() call succeeds.
job_card_status_history	Same shape, for job_cards.	Slice 2 adds the writers; for now the table exists empty.

PATTERN The Status-History Pattern

Each history table has columns (entity_id, from_status, to_status, action, reason, actor_user_id, created_at).

Inserted in the same transaction as the status change (per Doctrine 6).

Indexed on (entity_id, created_at) so retrieving the timeline for one request is constant-time.

Read by future Slice 2 features (timeline view on the request detail page) and by auditors investigating "when was this approved and by whom".

The code_counters table

DEFINITION code_counters

A two-column table: (scope VARCHAR, year SMALLINT, next_seq INT UNSIGNED). Primary key on (scope, year).

Backs the codeGenerator utility. Each row holds the next sequence number to issue for one (entity, year) pair.

Updated atomically with INSERT ... ON DUPLICATE KEY UPDATE next_seq = LAST_INSERT_ID(next_seq + 1). No row locks held across HTTP round-trips.

HOW TO THINK Counter table vs database SEQUENCE vs UUID

A database SEQUENCE (Postgres) is cleanest but MySQL 8 does not have native sequences with year-scoping.

A UUID is trivial but produces JR-A7F3-89D2-... which auditors hate.

A counter table with the atomic-increment idiom gives us the human-readable codes auditors want, with no contention up to several thousand QPS — far above what an internal MIS will ever see.

If we ever outgrow this technique we switch to Twitter's Snowflake or HashIDs at a single file change.

Database story closure

WHAT WE DID Database story closure

Discovered the real schema with read-only introspection — never assumed.

Wrote SCHEMA_PHASE6.md as the binding contract between canonical and real names.

Added eleven columns, six indexes, two history tables, one counter table — every change additive and idempotent.

Designed for covering reads (EXPLAIN shows "Using index") and atomic writes (no SELECT FOR UPDATE on the hot path).

Left every Phase 3 table intact. The seal holds.

Part V — Backend Module Architecture

Phase 6 adds three new backend modules (jobRequests, jobCards, lookups), two new utilities (auditLog, codeGenerator), one new piece of middleware (a real rowLevelScope to replace the Phase 4 stub), and one extension to an existing module (users gains /me/profile). This part walks the architecture of each, without source code.

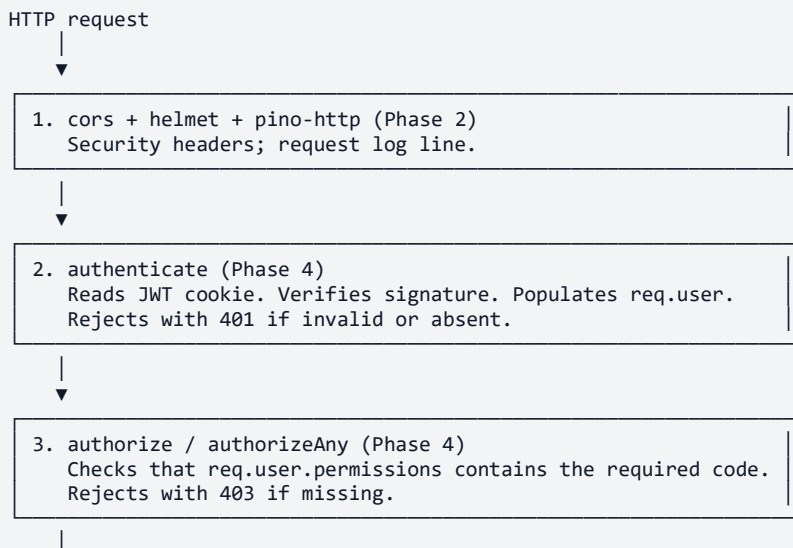
The backend after Phase 6

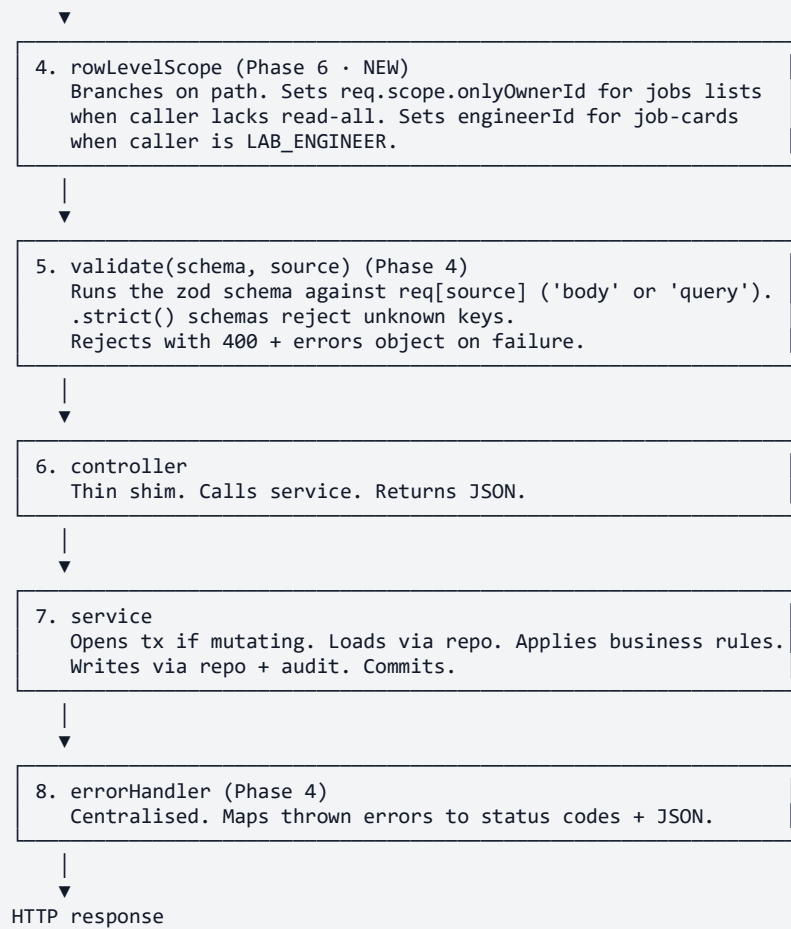
```

backend/src/
├── config/                ← Phase 2 (sealed)
├── middleware/
│   ├── authenticate.js    (Phase 4 · sealed)
│   ├── authorize.js       (Phase 4 · sealed)
│   ├── rowLevelScope.js   (Phase 6 · REPLACED – was a stub)
│   ├── validate.js        (Phase 4 · sealed)
│   └── errorHandler.js    (Phase 4 · sealed)
├── modules/
│   ├── auth/              (Phase 4 · sealed)
│   ├── users/             (Phase 4 · PATCHED – added /me/profile)
│   ├── equipment/         (Phase 5 · sealed)
│   ├── jobRequests/       (Phase 6 · NEW · 6 files)
│   ├── jobCards/          (Phase 6 · NEW · 5 files)
│   └── lookups/           (Phase 6 · NEW · 3 files)
├── utils/
│   ├── crypto.js          (Phase 4 · sealed)
│   ├── cookies.js         (Phase 4 · sealed)
│   ├── auditLog.js        (Phase 6 · NEW)
│   └── codeGenerator.js    (Phase 6 · NEW)
└── server.js              (Phase 6 · PATCHED – 3 app.use lines)
  
```

The request lifecycle in Phase 6

Every HTTP request that reaches a Phase 6 endpoint travels through the same chain. Understanding this chain is essential before reading any individual route file.





Module 1 — jobRequests

MODULE jobRequests

Owner of: the job_requests table, the job_request_status_history table, the request_code sequence.

Files: validators, stateMachine, repo, service, controller, routes (6 total).

Public surface: 3 endpoints — list, create-draft, submit.

Locked rules: BR-JR-06 (server-set submitter), T&C double-lock, audit pairing, choke-point pattern.

Future slices: add detail endpoint, approve, reject, assign-engineer endpoints + auto-create-job-card hook.

The three endpoints

Method · Path	Verb	Permission	Returns
GET /api/v1/job-requests	READ-list	job_request:read-own (loose) + scope	Paginated rows + pagination meta + applied filters
POST /api/v1/job-requests	CREATE-draft	job_request:create	New row id + request_code + status (DRAFT)
POST /api/v1/job-requests/:id/submit	TRANSITION DRAFT→SUBMITTED	job_request:create	Same row id + request_code + new status (SUBMITTED)

Why TWO endpoints for create-then-submit instead of one

DECISION Two-call create-then-submit · *create + submit as two verbs*

Decision: creating and submitting are two separate API calls, not a single "create as submitted" call.

Alternative considered: one POST with a "draft or submitted" flag.

Why two won: it cleanly separates two distinct verbs. Draft = "save my partial work, no audit row of submission yet." Submit = "I am ready, write the SUBMITTED transition with timestamp and TnC version."

It also makes the FE "Save as Draft" button a simple call to endpoint 1 with no extra coordination.

And it gives recovery semantics: if the second call fails (network blip), the draft survives and the user retries without re-typing.

What each layer in jobRequests is responsible for

Layer	Responsibility	Touches DB?
validators	Define createSchema, submitSchema, listQuerySchema as zod .strict() shapes. Reject unknown keys (BR-JR-06 enforcement point #1).	No
stateMachine	Encode allowed transitions as a table. Pure function transition() throws on illegal.	No
repo	Own all SQL. Alias real columns to canonical names on SELECT. Bind canonical to real on INSERT/UPDATE.	Yes
service	Open transaction. Load fresh actor snapshot. Call code generator. Insert via repo. Write audit row. Commit.	Yes (via repo + audit utility)
controller	Read req.user, req.body, req.query, req.params. Call service. Return JSON. Rethrow errors.	No
routes	Wire the express path to its middleware chain. No logic.	No

Module 2 — jobCards

MODULE jobCards

Owner of: the job_cards table, the job_card_status_history table, the card_code sequence.

Files (Slice 1): validators, repo, service, controller, routes (5 total — no state machine yet because Slice 1 is list-only).

Public surface: 1 endpoint — list.

Future slices: detail endpoint, state-machine module, transitions (ASSIGNED → IN_PROGRESS → COMPLETED → VERIFIED_CLOSED), observation entry, PDF generation.

HOW TO THINK Why Slice 1 has no jobCards.stateMachine.js file

No mutations land in Slice 1. The job_cards table is read-only from the API perspective.

A state machine without callers is dead code — adding it now would invite drift before Slice 2 wires the callers.

Slice 2 will add the file alongside the first mutation. The folder shape will then exactly mirror jobRequests.

Lesson: introduce a layer only when there is a caller for it. Speculative layers are the worst kind of premature optimisation.

Module 3 — lookups

MODULE lookups

Owner of: read-only access to small reference data the form needs (divisions list, equipment search results).

Files: repo, controller, routes (3 total — no state machine, no service layer because there is no business logic, just SQL).

Public surface: 2 endpoints — GET /lookups/divisions and GET /lookups/equipment/search?q=...

Permission gate: both endpoints require job_request:create. Reason: only users who can create requests need to populate the form's dropdowns and typeaheads.

TRADE-OFF Single tiny module vs scattering endpoints across larger modules

Alternative: put GET /divisions inside the (eventual) divisions module and GET /equipment/search inside the equipment module.

Why we chose a lookups module: these endpoints exist for the form, not for management of divisions or equipment. They have a different permission profile (job_request:create) than the underlying entity's CRUD endpoints would have.

Cost: a tiny module with three thin files. Worth it because the lookup permission stays focused, and future "form dropdown" endpoints have a natural home.

Utility 1 — auditLog

DEFINITION auditLog

A single-function module exporting writeAudit(conn, payload).

Inserts one row into audit_log with: entity name, entity id, action, actor user id, before-JSON, after-JSON, ip, user agent, created-at.

Crucial detail: accepts a connection parameter so the caller can pass the transaction connection. The insert lands in the same transaction as the state change.

Refuses to log "into the void" — there is no ambient connection. Forgetting to pass one is a loud error, not a silent no-op.

PATTERN The Audit Pairing Pattern

Step 1: service opens conn.beginTransaction().

Step 2: service calls repo to write the state change using conn.

Step 3: service calls writeAudit(conn, ...) using the same conn.

Step 4: service calls conn.commit().

Either both writes land or neither does. There is no partial outcome.

In the catch block: conn.rollback() is unconditional. Finally: conn.release() returns the connection to the pool.

Utility 2 — codeGenerator

DEFINITION codeGenerator

A two-function module exporting generateRequestCode(conn) and generateCardCode(conn). Both delegate to nextSequence(conn, scope) which performs the atomic-increment idiom against code_counters.

Returns a fully formatted string: JR-2026-0001 or JC-2026-0042.

Accepts an optional conn parameter so the sequence increment can join an existing transaction.

PATTERN The Atomic-Increment Idiom

INSERT INTO code_counters (scope, year, next_seq) VALUES (?, ?, LAST_INSERT_ID(1)) ON DUPLICATE KEY UPDATE next_seq = LAST_INSERT_ID(next_seq + 1).

On the first call for a (scope, year): inserts row with next_seq = 1, LAST_INSERT_ID() returns 1.

On every subsequent call: updates next_seq, LAST_INSERT_ID() returns the new value.

No SELECT FOR UPDATE. No application-level lock. No race condition possible because the entire increment is one statement.

Returns the issued sequence directly via mysql2's insertId on the result.

Middleware — rowLevelScope (replaced)

DEFINITION rowLevelScope (Phase 6 version)

Express middleware. Runs after authenticate + authorize. Sets req.scope to an object that downstream repo calls consult.

For /api/v1/job-requests: if the caller lacks job_request:read-all, sets req.scope.onlyOwnerId = req.user.userId. Otherwise sets {}.

For /api/v1/job-cards: if the caller's role is LAB_ENGINEER, sets req.scope.engineerId = req.user.userId. Otherwise sets {}.

For other paths: no-op.

Phase 4 had a stub that just called next(). Phase 6 replaces it with the real branching logic.

HOW TO THINK Why scope is set in middleware, not in the service

If every list service had to call hasPermission("read-all") and branch internally, the same six-line block would appear in every list endpoint. Drift would creep in within months.

Middleware lifts the branching into one place. Every list endpoint reads req.scope and trusts it.

It also makes the rule auditable: a single file owns "what does scope mean for path X?" — easy to review, easy to extend.

Trade-off: middleware reads the path, which couples the file to the URL design. We accept that because URLs are sealed at this point.

Existing module patch — users

MODULE users (Phase 6 patch)

Existing endpoint GET /api/v1/users/me returns id, employee_id, role, permissions. Unchanged.

New endpoint GET /api/v1/users/me/profile returns the full profile: full_name, designation, email, lab_phone, room_phone, division_id, division_code, division_name.

Used by the New Job Request form's Section 4 to auto-fill the Submitted-By identity and division.

Cached on the frontend for one hour (rarely changes). Invalidated on logout.

WHAT WE DID Backend architecture closure

Three new modules (jobRequests, jobCards, lookups) added without altering any existing module.

Two utilities (auditLog, codeGenerator) added in /utils — reusable for every future phase.

One middleware replaced (rowLevelScope) with no caller code changes — same signature, real behaviour.

One existing module gained one route (/me/profile) without touching its existing /me endpoint.

Three lines added to server.js. The Phase 4 + Phase 5 + Phase 6 stack is one server.

Part VI — The Job Request State Machine

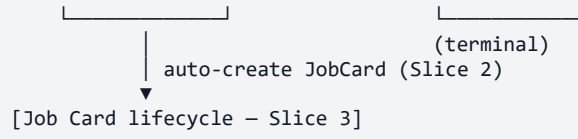
The state machine is the most important file in Phase 6. It is also the smallest. If you understand only one file in the entire backend, make it this one.

The states

State	Meaning	Slice that mutates into it
DRAFT	The user has saved partial work. No commitment.	Slice 1 (this).
SUBMITTED	The user has accepted T&C and the request is awaiting approval.	Slice 1 (this).
REJECTED	Lab In-Charge declined the request with a reason.	Slice 2.
ASSIGNED	Lab In-Charge approved + assigned an engineer. Job Card auto-created.	Slice 2.
IN_PROGRESS	Job Card transitioned out of ASSIGNED — engineer is working on it.	Slice 3 (via job card state machine).
COMPLETED	Job Card observations entered; awaiting verification.	Slice 3.
VERIFIED_CLOSED	Lab In-Charge verified completion; equipment last_cal_date updated.	Slice 3.
REOPENED	Verification failed; back to the engineer.	Slice 3.
CANCELLED	Owner withdrew a draft. Terminal.	Slice 1 (this, but no UI yet).

The transitions for Slice 1





The transition table

From state	Action	To state	Permission required	Must be owner?
DRAFT	submit	SUBMITTED	job_request:create	Yes
DRAFT	cancel	CANCELLED	job_request:create	Yes
SUBMITTED	approve	ASSIGNED	job_request:approve	No (LIC/SA only)
SUBMITTED	reject	REJECTED	job_request:reject	No (LIC/SA only)
SUBMITTED	assign	ASSIGNED	job_request:assign-engineer	No (LIC/SA only)
ASSIGNED	(any)	(any)	— hand-off to Job Card lifecycle —	—
REJECTED	(any)	(any)	— terminal —	—
CANCELLED	(any)	(any)	— terminal —	—

HOW TO THINK Read the table as a contract, not a list

The presence of a (from, action) pair declares it legal. The absence declares it forbidden.

An action against a state not in the table throws `ILLEGAL_TRANSITION` (HTTP 409). Always.

The permission column says "who can invoke this transition" — checked at the choke-point, redundant with route-level `authorize()` but defence in depth.

The "must be owner" column captures actor-context: a submit on a draft is only legal for the user who created the draft, regardless of permissions.

The function signature, conceptually

PATTERN `transition(currentState, action, actor, context)`

`currentState`: the entity's status loaded fresh from the DB inside the transaction. Trust no client-supplied state.

`action`: a verb string ("submit", "approve", etc.) from the controller, not from the user.

`actor`: an object containing at least `{ userId, permissions[] }`. Populated by the authenticate middleware.

`context`: optional. For Slice 1 the only relevant field is `isOwner`. Slice 2 will add `reason for reject()`.

Returns: `{ newState }` on success. Throws: `TransitionError` with one of three codes — `ILLEGAL_TRANSITION` (409), `NOT_OWNER` (403), `FORBIDDEN` (403).

Side effects: none. It is a pure function. The service layer is responsible for applying the new state to the DB and writing the audit row.

The T&C double-lock at submit

PRINCIPLE Two independent layers gate submit on T&C acceptance

Frontend layer: the Submit button is disabled while accepted < 6/6. The user cannot click.

Backend layer: the submitSchema requires tnc_accepted to be the literal value true. Any other value (false, missing, "yes") fails validation with HTTP 400.

Result: even if a malicious client re-enables the button in DevTools and sends false, the server rejects.

HOW TO THINK Why tnc_accepted is z.literal(true), not z.boolean()

z.boolean() would accept both true and false — defeating the purpose of validation at this layer.

z.literal(true) accepts only true. False is rejected with a clear "Invalid literal value, expected true" message.

This subtlety is exactly the kind of one-character difference that defence in depth depends on. Get it wrong and the BE check evaporates.

The submit transaction, end to end

```

Controller receives POST /api/v1/job-requests/:id/submit body={tnc_accepted:true,tnc_version:"v1"}
↓
Service.submit(id, body, actor, requestMeta)
├─ if body.tnc_accepted !== true: throw 400 (defence in depth)
├─ const conn = await pool.getConnection()
├─ await conn.beginTransaction()
├─ existing = repo.findById(id)           ── load fresh state inside tx
├─   if !existing: throw 404
├─ const isOwner = existing.submitted_by_user_id === actor.userId
├─ { newState } = transition(existing.status, 'submit', actor, { isOwner })
├─   ↑↑↑ choke-point. throws if illegal ↑↑↑
├─ repo.updateStatus(conn, id, newState, {
├─   tnc_accepted_at: now,
├─   tnc_version: body.tnc_version || 'v1',
├─   submitted_at: now
├─ })
├─ repo.insertStatusHistory(conn, id, existing.status, newState, 'submit', actor.userId)
├─ writeAudit(conn, {
├─   entity: 'job_request', entityId: id, action: 'SUBMIT',
├─   actorUserId: actor.userId,
├─   before: { status: existing.status },
├─   after: { status: newState, tnc_version: body.tnc_version || 'v1' },
├─   ip: requestMeta.ip, userAgent: requestMeta.userAgent
├─ })
├─ await conn.commit()           ── ALL three writes commit together
├─ conn.release()
↓
Controller returns 200 with { data: { id, request_code, status: 'SUBMITTED' } }

```

What survives a failure at each step

Failure at	Status row?	History row?	Audit row?	User sees
Validator (TnC missing)	No	No	No	HTTP 400 with field-level error.
repo.findById (not found)	No	No	No	HTTP 404. No state changed.
transition() (illegal/forbidden)	No	No	No	HTTP 409 or 403. No state changed.
repo.updateStatus (SQL error)	No	No	No	Tx rolled back. HTTP 500. Pino log line.
repo.insertStatusHistory (SQL)	No	No	No	Tx rolled back. HTTP 500.

Failure at	Status row?	History row?	Audit row?	User sees
writeAudit (SQL error)	No	No	No	Tx rolled back. HTTP 500.
conn.commit (rare — disk full)	No	No	No	Tx aborted. HTTP 500.
After commit (success)	YES	YES	YES	HTTP 200. All three rows are visible.

★ The all-or-nothing invariant

Either every row in the table marked "YES" is written, or none is.

There is no intermediate state where status_history has a row but audit_log does not.

There is no intermediate state where the status field is updated but the history is missing.

This is the audit-pairing rule, made physical by the database transaction.

WHAT WE DID State machine deep dive closure

Mapped every state and every transition into one table that is also the contract.

Located the entire state-change pipeline in one transaction so failure modes are bounded.

Made the T&C double-lock physical: client gate + server literal-true check.

Confirmed that the all-or-nothing invariant holds at every failure point.

Part VII — Frontend Architecture & Data Flow

The frontend mirrors the backend in three ways: schemas (zod on both sides), permissions (same codes on both sides), and naming (canonical names appear in the React tree, not raw DB names). Phase 6 adds three pages, eight components, three API hook files, and two schema files. Everything else — auth, routing, the API client, the Tailwind palette, the layout shell — already existed.

The frontend after Phase 6

```

frontend/src/
├── App.jsx                                     (Phase 5 • PATCHED – 3 new routes)
├── lib/
│   ├── api-client.js                         (Phase 4 • sealed)
│   ├── auth-context.js                      (Phase 4 • sealed)
│   └── permissions.js                       (Phase 5 • PATCHED – 2 new nav items)
├── components/
│   ├── Layout.jsx                          (Phase 4 • sealed)
│   └── ProtectedRoute.jsx                   (Phase 4 • sealed)
├── api/
│   ├── jobRequests.api.js                  (Phase 6 • NEW)
│   ├── jobCards.api.js                    (Phase 6 • NEW)
│   └── lookups.api.js                      (Phase 6 • NEW)
├── schemas/
│   ├── jobRequestSchemas.js               (Phase 6 • NEW)
│   └── jobCardSchemas.js                  (Phase 6 • NEW)
├── pages/
│   ├── jobRequests/
│   │   ├── JobRequestList.jsx              (Phase 6 • NEW)
│   │   └── JobRequestNew.jsx               (Phase 6 • NEW)
│   │   └── components/
│   │       ├── StatusPill.jsx
│   │       ├── PriorityLabel.jsx
│   │       ├── PaginationBlock.jsx
│   │       ├── JobRequestFilters.jsx
│   │       └── JobRequestTable.jsx
│   │       └── form/
│   │           ├── tncContent.js
│   │           ├── Section1JobType.jsx
│   │           ├── Section2Equipment.jsx
│   │           ├── Section3Accessories.jsx
│   │           ├── Section4SubmittedBy.jsx
│   │           └── Section5TermsAndConditions.jsx
│   └── jobCards/
│       ├── JobCardList.jsx
│       └── components/JobCardTable.jsx

```

Doctrine 9 — Mirror the backend schema, never invent your own

PRINCIPLE Frontend zod schemas are line-for-line mirrors of backend schemas

Same field names, same constraints, same enums, same maxima.

Same .strict() discipline: the form cannot send a field the backend would reject.

When the backend schema changes, the frontend schema changes in the same PR. Drift is a code-review failure.

HOW TO THINK Why mirroring matters more than DRY

A naive engineer would try to share schemas across BE and FE via a published package. That introduces a build dependency, a versioning headache, and risk of FE shipping ahead of BE.

Mirroring duplicates code — but the duplication is shallow (schema files are small) and reviewable in one diff.

The trade-off favours simplicity: a five-line schema duplicated is cheaper than a microservices-grade contract bus.

Doctrine 10 — URL is the source of truth for list state

PRINCIPLE Every filter, sort, and pagination value lives in the URL

The list page reads `useSearchParams()` to derive its filter object. The filter object is what the API hook receives.

When a filter changes, the page calls `setSearchParams()` — the URL updates, the query refetches, the table re-renders.

Result: bookmarking, deep linking, back/forward buttons, and refresh-on-current-view all work for free.

HOW TO THINK Why `useState` would be wrong here

`useState` keeps state in the component. Refresh wipes it. The user loses their filters every reload.

`useState` also breaks deep linking: "send me the URL of that view you found yesterday" cannot be answered.

`useSearchParams` is React-Router's answer to exactly this problem. It is also the standard pattern from the Phase 5 equipment list — we are reusing a working idiom, not inventing one.

Doctrine 11 — React Query owns server state

PRINCIPLE Server data lives in React Query, never in component state

List endpoints use `useQuery` with `keepPreviousData` so filter changes do not flash empty.

Mutations use `useMutation` and invalidate the relevant query key on success.

Cache lifetime is tuned per resource: divisions = 5 min, equipment search = 30 s, profile = 1 hour, list pages = 30 s.

The cache-invalidation contract

User action	What invalidates
User saves a new draft via <code>useCreateDraft</code>	The "job-requests" query key — every list view refetches.
User submits a draft via <code>useSubmitRequest</code>	The "job-requests" query key — the new SUBMITTED row appears.
User logs out	Every query key — entire cache purged.
User switches account (rare)	Every query key — refetch with new auth.
User idle for > 30 s and clicks a sort header	The stale list refetches automatically (staleTime = 30 s).

TRADE-OFF 30-second stale time vs immediate freshness

A user reading the list page sees data up to 30 seconds old. For an internal MIS this is acceptable; for a stock-ticker it would not be.

In exchange we avoid hammering the backend with one query per keystroke or click. A team of 50 lab engineers all watching the list cost the server roughly two queries per minute, not two per second.

If a particular workflow ever needs sub-second freshness (real-time approval dashboard), it can override staleTime locally without changing the global default.

Doctrine 12 — Components are stateless and composable**PRINCIPLE Each component owns one concern**

StatusPill: render a colored badge for one status string. No data fetching, no callbacks beyond presentation.

PriorityLabel: render a styled label for one priority value.

PaginationBlock: render the page selector. Receives state, emits onChange.

JobRequestFilters: render filter inputs. Receives current filter object, emits new one.

JobRequestTable: render rows. Receives rows + loading flag.

Form Sections (1-5): render their own fields. Receive the form object from react-hook-form; do not own form state.

JobRequestList / JobRequestNew: the pages. Orchestrate everything below.

Data flow — the new request form



HOW TO THINK Why the form lives in one page component but its sections are siblings

A monolithic 400-line form file becomes unreadable. Splitting into five sibling components keeps each under 100 lines.

The shared state — react-hook-form's form object — is passed down as a prop. Each section uses register() and setValue() against it.

The T&C checkboxes are NOT in the react-hook-form object. They are a parent useState, because they are pre-commit UI state, not data to be sent to the server.

This split mirrors the backend layering: data that travels to the server is in the form object; pure UI gating is in useState.

Permission gating on the frontend

Surface	Gate	Behaviour when missing
Sidebar "Job Requests" link	job_request:read-own	Item hidden from sidebar.

Surface	Gate	Behaviour when missing
Sidebar "Job Cards" link	job_card:read-list	Item hidden from sidebar.
"+ New Job Request" CTA	job_request:create	Button hidden from list header.
/job-requests/new route	job_request:create	Redirect to /403 via ProtectedRoute.
/job-requests route	job_request:read-own	Redirect to /403.
/job-cards route	job_card:read-list	Redirect to /403.
Approve / Reject buttons	job_request:approve / job_request:reject	Will be added in Slice 2 — gated similarly.

PRINCIPLE UI gates hide; server gates enforce

The sidebar hides items the user lacks permission for — this is courtesy, not security.

Removing a UI gate (DevTools enabling a hidden button) does not grant access — the server still rejects with 403.

A real security control is always server-side. UI gates exist to spare users the experience of clicking a button only to see "Forbidden".

WHAT WE DID Frontend architecture closure

Mirrored backend schemas on the frontend with no shared package — drift caught at PR review.

Made URL the source of truth for list state — bookmarking, deep links, refresh all work.

Pushed all server state into React Query with explicit cache lifetimes per resource.

Made every component stateless and composable; pages orchestrate, components present.

Confirmed that UI gates and server gates are independent defences against unauthorised actions.

Part VIII — The Three Screens (Anatomy)

Each of the three Slice 1 screens has a deliberate composition. This part dissects them — what they show, why those choices, what they consciously omit, and how they interact with the backend.

Screen 1 — /job-requests (list)

Anatomy

Job Requests [+ New Job Request]								← header bar
Manage equipment calibration and maintenance requests								
[search box (q)] [Type ▼] [Status ▼]								← filter row 1
[Advanced Filters] [Export (disabled)] Showing N of N								← filter row 2
Job ID	Equipment	Type	Div	Submitted	Date	Pri	Status	
JR-...-001	Multimeter Fluke	Cal.	OAD	Patel R.	12/05	Med	Pending	← rows
JR-...-002	Oscilloscope	Repair	SED	Sharma A.	11/05	High	Approved	
...								
Page 1 of 4 Rows: 25 ▼ [Prev] [1] [2] [3] [4] [Next]								← pagination

What each region does

Region	Component	Source of data	Notes
Header bar	(inline in page)	Static + auth-context for CTA visibility.	CTA hidden if no <code>job_request:create</code> .
Filter rows	JobRequestFilters.jsx	URL → <code>useSearchParams</code> → filter object.	Search uses <code>defaultValue</code> + <code>onChange</code> , not <code>value</code> , so React does not re-render on every keystroke.
Table	JobRequestTable.jsx	<code>useJobRequestsList(filters)</code> → rows + total.	Skeleton on <code>isLoading</code> ; empty-state when <code>rows.length === 0</code> .
Pagination	PaginationBlock.jsx	Same query result's pagination meta.	Page-size selector is part of the component; emits <code>onPageSizeChange</code> .

The pagination block — design notes

PATTERN The "1 2 ... 99 100" sliding window

Always show the first 2 page numbers and the last 2 page numbers.

Always show the current page and one neighbour on each side.
When the gaps between these clusters are > 1 , insert an ellipsis.
Result: at 100 pages the block never grows wider than nine cells. Always clickable, always readable.
Below 7 pages, show every page — the windowing is unnecessary.

HOW TO THINK Why the table is a pure presentation component

JobRequestTable.jsx receives rows + loading flag. It does not call the API. It does not own filters. It does not invalidate anything.

This means it can be tested with hard-coded data in Storybook, dropped into a different page, or replaced wholesale (e.g. with a virtualised table) without ripple effects.

The lesson: any component that does both presentation and data-fetching is a layering violation. Split the two and you can reason about each independently.

Screen 2 — /job-requests/new (form)

Anatomy

← Back to Job Requests

New Job Request

Submit a new calibration, repair, or registration request

1. Job Type Selection Job Category * [TME / FPE] Job Type * [CALIBRATION / REPAIR / REGISTRATION]
2. Equipment Details Equipment ID * [typeahead search → fills fields below] Equipment Name * Make / Model / Serial / Type Options / Description (textarea)
3. Accessories (optional) [+ Add Accessory] rows: type, name, serial
4. Submitted By Name * (disabled, from /me/profile) SAC Employee ID *, Designation *, Email * (all disabled) Lab Phone, Room Phone (editable, pre-filled) Division * Subsystem Project Complaint Description * (textarea, min 10 chars) Remarks [] Equipment sent after repair
5. Terms and Conditions [] T&C 1: equipment details are accurate... [] T&C 2: equipment delivered in working condition... [] T&C 3: calibration timeline begins after receipt... [] T&C 4: coordinate with assigned engineer... [] T&C 5: damaged-or-beyond-repair handling... [] T&C 6: urgency requests need authorisation... ⚠ You must accept all (0/6 accepted)
[Cancel] [Save as Draft] [Submit]

Five sections — five purposes

Section	Purpose	Why it is its own component
1	Capture the abstract category and type of the job.	Two simple selects with shared placeholder logic.
2	Identify the equipment by ID or by typed details.	Typeahead search is non-trivial; isolating it keeps the parent page simple.
3	List optional accessories.	useFieldArray adds dynamic complexity that belongs in its own file.
4	Auto-fill the requester and capture division/project.	Mixes profile data (read-only) with editable fields; isolation makes the BR-JR-06 boundary visible in one place.
5	Gate the Submit button via T&C acceptance.	Holds non-form state (T&C checkbox state) separately from the form object.

Section 4 — the most important section in Slice 1

★ Section 4 is the visible enforcement of BR-JR-06

The four identity fields (Name, SAC ID, Designation, Email) render as disabled inputs.

The four field names do NOT appear in the form schema. react-hook-form's register() is not called for them.

A user who enables them in DevTools and types a value sees no effect — the values are never read into the submission payload.

A user who appends submitted_by_user_id to the network payload via Fetch directly will be rejected by the .strict() backend schema.

Two independent layers, each sufficient on its own.

Save-as-Draft vs Submit — the contract

Button	What it does	T&C gate?	Schema validation?
Save as Draft	Calls POST /job-requests with the current values. Saves a DRAFT row.	No	Yes — but allows missing optional fields.
Submit	Calls POST /job-requests to create the DRAFT, then POST /:id/submit.	Yes (6/6)	Yes — full schema enforced.

HOW TO THINK Why "Submit" creates the draft first instead of a single create-submitted call

The natural impulse is to send everything in one POST with a "submitted: true" flag.

But that ties two distinct verbs into one call. If the second step (the transition) fails for any reason (transient network, server restart), the user has lost their data.

Two calls preserves recovery: if the second fails, the draft still exists. The user retries the submit step alone, without re-typing.

The UX cost: a 50ms second round-trip. The DX/audit-grade benefit: significant.

Screen 3 — /job-cards (list)

Anatomy

Job Cards Track and manage job execution and progress						
[search box] [Status ▼] [Advanced Filters] [Export (disabled)] Showing N of N						
Job Card ID	Job Request	Equipment	Engineer	Status	Start	Due
JC-...-001	JR-...-001	Multimeter	Sharma A.	In Progress	13/05	20/05
...						
Page 1 of 2 Rows: 25 ▼ [Prev] [1] [2] [Next]						

Differences from Screen 1

Aspect	Screen 1 (Requests)	Screen 3 (Cards)
New-record CTA	Visible to Normal/Eng/LIC/SA.	Absent — cards are auto-created from approved requests.
Filters	q, type, status, priority, division.	q, status (no type — cards inherit from request).
Row-level scope	submitted_by_user_id for Normal.	assigned_engineer_id + queue for LAB_ENGINEER.
Parent reference	—	Each row shows JR-2026-XXXX from a LEFT JOIN.
Permission to view	job_request:read-own (everyone).	job_card:read-list (NOT Normal Users).

HOW TO THINK Why the cards list reuses PaginationBlock from the requests folder

PaginationBlock has no Job-Request-specific knowledge. It takes (page, totalPages, pageSize, onPageChange, onPageSizeChange) — that's it.

Reusing it across pages avoids a second component drift target. Improvements (keyboard navigation, jump-to-page input) land in one place.

The "from the requests folder" awkwardness is a sign we should move the component to a shared folder in Slice 2 — a tiny refactor.

What the three screens DO NOT have (intentionally)

! Deferred to later slices

No detail page — clicking a row would navigate to /job-requests/:id, which does not exist yet.

No inline status changes — every status is read-only in Slice 1.

No approve / reject buttons — Lab In-Charge sees the same view as Lab Engineer.

No CSV / Excel export — the Export button exists and is disabled (telegraphs intent, prevents user complaints about a missing feature).

No bulk-select checkboxes — bulk actions are a Slice 2 design conversation.

WHAT WE DID Three-screens closure

Built one read-list pattern, one write-form pattern, and one parent-child list pattern.

Made BR-JR-06 visible in Section 4: identity fields disabled, field names absent from the schema.

Disabled the Export button instead of hiding it — users know it is coming, the team knows what to build next.

Reused PaginationBlock across both list pages — one component, one bug surface, one improvement target.

Part IX — Verification Methodology

A slice is "done" when twelve acceptance criteria pass. Each criterion is a business statement, not an implementation detail. Each maps to one smoke step, executable in under a minute. This part describes the methodology, not the commands (those live in the companion build guide).

The acceptance criteria, sealed

ID	Statement (what we are claiming)	Why this matters
A1	A Normal User sees only the job requests they themselves submitted.	BR-VIS-01 enforcement. Privacy law and audit trust.
A2	A Lab In-Charge sees every job request regardless of submitter.	Operational visibility for approvers.
A3	A View-Only user cannot create a job request.	Read-only roles must stay read-only at the server.
A4	The Submitted-By identity is set server-side; clients cannot override.	BR-JR-06. Audit integrity.
A5	Submit is rejected if T&C is not literally true.	T&C double-lock at the BE layer.
A6	Every successful state change writes exactly one audit row in the same tx.	BR-AUD-01. Audit pairing.
A7	Sort parameters outside the whitelist are rejected as 400 errors.	SQL injection prevention at the validator layer.
A8	Pagination math is correct: total, total_pages, page, page_size.	List UX correctness; trust in the page count.
A9	Filters survive page reload and copy-paste of the URL.	Bookmarking, sharing, refreshing.
A10	EXPLAIN on the default list query uses idx_jr_list_default and is "Using index".	Performance baseline; covering reads.
A11	p50 latency on the default list under 50 ms on a warm DB.	Quantitative perf target from FINAL-DESC §11.
A12	View-Only can view job cards; Normal cannot.	Permission gate on /job-cards.

How to read this list

HOW TO THINK Acceptance criteria are claims, not tests

Each line is a sentence the system either satisfies or violates. "A1: Normal sees own only" is true or false at any given moment.

The smoke test is the proof — it generates evidence that the claim holds at a specific point in time.

A green smoke run for A1 today does not prove A1 forever. It proves A1 at the commit that was tested.

Therefore: smoke tests are run before every merge, and re-run on the green build that ships. The proof is the artifact of the build, not the artifact of the developer.

The smoke-test methodology

PRINCIPLE Smoke tests prove behaviour, not implementation

A smoke test calls the public API. It does not poke internal modules.

A smoke test reads observable outputs: HTTP status, response body shape, audit_log rows, EXPLAIN output, response latency.

A smoke test passes or fails for the same reason a human would: the observable contract is met or it is not.

A smoke test never imports backend code. If it did, refactoring would break smoke without breaking the system.

The twelve smoke steps, conceptually

Smoke #	Acceptance	Mechanism	Pass condition
1	A1	Login as Normal User. List job requests. Inspect submitter names.	All rows have the Normal User's own name as submitter.
2	A2	Login as Lab In-Charge. List job requests. Compare total to Normal's.	LIC total >= Normal total.
3	A3	Login as View-Only. POST /job-requests with valid body.	HTTP 403 with permission error.
4	A4	POST with submitted_by_user_id field in body.	HTTP 400 — "Unrecognized key" because schema is .strict().
5	A5	Create a draft. POST /:id/submit with tnc_accepted: false.	HTTP 400 — literal-value error.
6	A6	Inspect audit_log after a successful create-then-submit.	Exactly two rows: CREATE_DRAFT then SUBMIT.
7	A7	GET list with ?sort=password.	HTTP 400 — sort is not in the enum.
8	A8	GET list with various page/page_size combinations.	pagination object is internally consistent (page <= total_pages).
9	A9	Open the list, set filters, refresh.	Filters remain visible in URL and applied to the table.
10	A10	Run EXPLAIN on the default list SELECT in MySQL CLI.	key = idx_jr_list_default; Extra includes "Using index".

Smoke #	Acceptance	Mechanism	Pass condition
11	A11	Time 20 list calls; sort; report the 10th value.	p50 < 50 ms on a database with 10k+ rows.
12	A12	Login as Normal. Call /job-cards. Login as View-Only. Call /job-cards.	Normal: 403. View-Only: 200.

Pre-flight checks before smoke

HOW TO THINK Why pre-flight catches more bugs than smoke does

Smoke tests assume the stack is alive. They cannot tell you why curl fails to connect — only that it fails.

Pre-flight checks verify Node version, MySQL reachability, environment variables, port availability. Each check fails loudly with a specific message.

Five minutes of pre-flight saves an hour of "why is curl timing out" diagnosis.

Pre-flight is also when you confirm the right database is targeted. Smoke against the wrong DB destroys staging in minutes.

The pre-flight checklist

#	Check	How	Pass condition
1	Node version is $\geq 18.0.0$	<code>node -v</code>	Major version is 18, 20, or 22.
2	npm is installed	<code>npm -v</code>	Outputs a version.
3	MySQL CLI is installed	<code>mysql --version</code>	Outputs MySQL Ver 8.x.
4	MySQL server is reachable	<code>mysql -u DEV_USER -p -e "SELECT 1"</code>	Returns 1 row.
5	Target DB exists and is the right one	<code>mysql -e "USE cmcmis_dev_db; SELECT DATABASE()"</code>	Outputs cmcmis_dev_db.
6	Phase 6 migrations applied	<code>SHOW INDEX FROM job_requests LIKE 'idx_jr_list_default'</code>	Returns one row.
7	.env files exist and are populated	<code>test -s backend/.env && test -s frontend/.env</code>	Both non-empty.
8	Backend port is free	<code>lsof -i :3000</code>	No matching process.
9	Frontend port is free	<code>lsof -i :5173</code>	No matching process.
10	At least 4 seed users exist (one per role)	<code>SELECT role, COUNT(*) FROM users GROUP BY role</code>	Each tested role has ≥ 1 user.

WHAT WE DID Verification closure

Treated each acceptance criterion as a permanent claim, with one smoke step as evidence.

Separated pre-flight (environment) from smoke (behaviour) so failures are localised to the right layer.

Anchored smoke tests on observable public behaviour, not internal implementation.

Made the twelve-step list short enough to run after every merge, exhaustive enough to ship on.

Part X — Operational Readiness

Slice 1 ships when all twelve acceptance criteria pass. Shipping is more than passing tests — it includes tagging, monitoring, rollback discipline, and forward-looking notes for Slice 2.

Deployment discipline

Stage	Action	Owner
Pre-deploy	Run all 12 smoke tests on the deploy candidate.	Engineer.
Pre-deploy	Apply migrations 0010, 0011, 0012 to staging. Confirm SHOW INDEX result.	Engineer.
Pre-deploy	Tag the commit: git tag phase6-slice1-rc1 (release-candidate-N).	Engineer.
Deploy	Roll out backend first. Confirm /healthz returns 200.	Engineer.
Deploy	Roll out frontend. Confirm /job-requests loads as Normal User.	Engineer.
Post-deploy	Smoke #1 + #5 + #6 against the live system within 5 minutes of deploy.	Engineer.
Post-deploy	Tag the live commit: git tag phase6-slice1-green.	Engineer.
Post-deploy	Note the audit_log row count baseline so future drift is detectable.	Engineer.

What to monitor in the first 72 hours

Signal	Healthy range	Action if out of range
POST /job-requests p99 latency	< 200 ms	Inspect codeGenerator counter contention. EXPLAIN the INSERT.
GET /job-requests p50 latency	< 50 ms	Re-run EXPLAIN. Verify idx_jr_list_default is being used.
audit_log row growth rate	~2 rows per submitted request	Investigate if ratio < 1.5 — audit row may be missing for some state changes.
4xx error rate on /job-requests	< 5%	Spike usually means a client schema drifted. Re-check FE/BE schema parity.
5xx error rate	< 0.1%	Read the pino log line for the stack trace. Most likely a missing column in the canonical map.
Number of DRAFT rows older than 7 days	< 10% of total	High value means users abandon drafts. Track UX friction in Slice 2.
Number of REJECTED requests with no reason	0	If non-zero, reject UI (Slice 2) is bypassing the reason field.

Rollback strategy

PRINCIPLE Forward-rollback, not backward-rollback

Because every Phase 6 migration is additive and idempotent, "rolling back" is a misnomer — the previous version still runs against the same database.

To revert: deploy the previous backend image. The new columns/indexes sit unused. The old endpoints continue to work.

No DROP statements are written for rollback. We never undo schema; we ignore it.

If the new code is found to be data-corrupting (it should not be — but if), the action is: deploy the previous image, then write a forensic SQL script to repair the bad rows.

Forward-looking notes for Slice 2

i What Slice 2 will add

Job Request detail page (read-only) at /job-requests/:id.

Approve / Reject UI for Lab In-Charge on that detail page.

Reject reason capture + storage in job_request_status_history.reason.

The auto-create-job-card hook: when approve fires, a new row in job_cards is created in the same transaction.

jobCards.stateMachine.js — the state machine for the card lifecycle (ASSIGNED → IN_PROGRESS → COMPLETED → VERIFIED_CLOSED → REOPENED).

Re-assign engineer (a side transition that does not change status).

i What Slice 3 will add

Job Card detail page with observation entry forms.

PDF generation for completed job cards (calibration certificates).

The verify-close hook that updates equipment.last_cal_date when status moves to VERIFIED_CLOSED.

Re-open flow from VERIFIED_CLOSED → IN_PROGRESS when verification fails.

Notifications: emails on key transitions (approved, assigned, completed).

WHAT WE DID Operational readiness closure

Defined the deploy ritual: pre-deploy smoke, migrate, deploy backend, deploy frontend, post-deploy smoke.

Established seven monitoring signals with healthy ranges and concrete actions.

Codified forward-rollback as the only rollback strategy — schema is never undone.

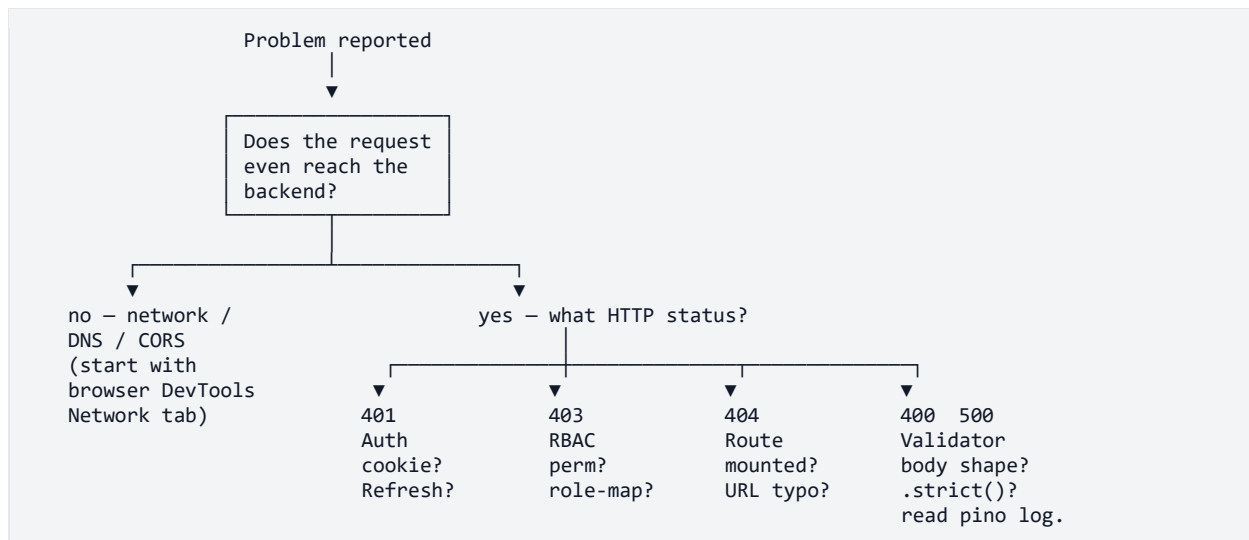
Scoped Slice 2 and Slice 3 explicitly so no Slice 1 work accidentally bleeds into the next.

Part XI — Troubleshooting Doctrine

When something breaks, do not guess. Walk the decision tree. The vast majority of Phase 6 bugs land in one of seven buckets. The right diagnosis is almost always faster than the right fix.

“Log first. Schema second. Permissions third. Code last.”

The decision tree



The seven buckets, expanded

Bucket 1 — 401 Unauthorized

HOW TO THINK When every call returns 401

Open browser DevTools. Network tab. Click a failed request. Inspect the Headers panel.

Question one: was a Cookie header sent? If no → frontend axios is missing withCredentials: true.

Question two: was a fresh login attempted? If no → the access token may have expired (15 min default). The refresh interceptor should renew automatically; if it does not, that interceptor is missing or broken.

Question three: did the backend restart since the .env was edited? Secrets load at boot — restart fixes silent failures.

Bucket 2 — 403 Forbidden

HOW TO THINK When the right user gets the wrong door slammed

curl /api/v1/users/me with the user's cookies. Inspect the permissions array in the response.

Is the required code present? If no → role_permissions seed is missing rows. Re-run the Phase 3 seed.

Is the code spelled correctly in the routes file vs the seed? Mismatch by even one character produces silent 403.

Is the role assigned to the user? Check users.role and confirm the role exists in role_permissions.

Bucket 3 — 404 Not Found

HOW TO THINK When the URL the client uses does not exist on the server

Check server.js: is there an app.use line for the new route group? Forgetting Step 6 of the build is the most common cause.

Check the path spelling. Express is case-sensitive and dash-sensitive. /job-requests vs /jobRequests vs /job_requests are three different paths.

Restart the backend after editing server.js. Node does not hot-reload changes to mounted routes.

Bucket 4 — 400 Bad Request

HOW TO THINK When validation rejects something that looks fine

Read the response body. The validator returns { errors: { field: "message" } }. The field name tells you exactly what zod rejected.

"Unrecognized key" → you sent a field the schema does not declare. Because the schema is .strict(), this is intentional. Remove the field. (Note: this is how BR-JR-06 is enforced.)

"Invalid literal value, expected true" → tnc_accepted was false or missing. The form must reach 6/6 before submit.

"Expected number, received string" → a query param wasn't coerced. listQuerySchema uses z.coerce.number() for exactly this reason. If a custom schema misses coerce, fix the schema.

Bucket 5 — 500 Internal Server Error

PITFALL Do not guess. Read the pino log line.

pino emits a structured JSON line on every error. It contains the stack trace, the request id, the path, and the user id.

The stack trace points at a file and a line. 90% of the time the file is a repo and the cause is a column-name mismatch — your canonical name does not match the real DB column.

Fix path: open SCHEMA_PHASE6.md, find the real name, add an alias in the repo SELECT (real_name AS canonical_name).

Less common: ER_BAD_FIELD_ERROR means the column does not exist at all. Either run migration 0010 or document the column as missing in the canonical map.

Bucket 6 — Pagination shows the wrong total

HOW TO THINK When data row count and total disagree

The data SELECT and the COUNT SELECT must use the same WHERE clause. If they diverge, total reflects a different filter set than data.

Inspect repo.findList — it builds where[] and args[] once and uses them for both the data query and the count query.

rowLevelScope additions (onlyOwnerId, engineerId) must be applied to BOTH queries. Forgetting one is a privacy bug, not just a count bug.

Bucket 7 — Form submits but new row does not appear in list

HOW TO THINK When mutation succeeds but the UI is stale

useCreateDraft and useSubmitRequest must call `queryClient.invalidateQueries({ queryKey: ["job-requests"] })` in `onSuccess`. Without it, React Query keeps the cached list.

If the mutation hook is correct, check that the list and the mutation use the same query key. `["job-requests", filters]` and `["jobRequests", filters]` are different keys.

If both are correct, the new row really did not save. Query audit_log: if you see both `CREATE_DRAFT` and `SUBMIT` rows, the data is in the DB and the bug is the FE cache. If you see only `CREATE_DRAFT` or neither, the submit step failed silently — check the network tab for the second call.

When in doubt

★ The golden order of investigation

Step 1: Read the log. The pino line tells you more than five guesses.

Step 2: Check the schema. Does the column exist? Is the canonical map accurate?

Step 3: Check the permissions. What does `/users/me` say the user has?

Step 4: Read the code. Only after the first three are exhausted is the code likely the bug.

WHAT WE DID Troubleshooting closure

Mapped seven buckets that cover the vast majority of failures.

Established the "log → schema → permissions → code" investigation order.

Made each bucket's mental model explicit so engineers diagnose, not guess.

Part XII — Appendices & Decision Log

Appendix A — Complete Permission Reference

Code	N	V	E	L	S	Where used
dashboard:view	✓	✓	✓	✓	✓	Phase 4 dashboard
equipment:read-list	✓	✓	✓	✓	✓	Phase 5
equipment:read-detail	✓	✓	✓	✓	✓	Phase 5
equipment:create	X	X	X	✓	✓	Phase 5
job_request:create	✓	X	✓	✓	✓	POST /job-requests + lookups
job_request:read-own	✓	✓	✓	✓	✓	GET /job-requests + nav item
job_request:read-all	X	✓	✓	✓	✓	Bypasses rowLevelScope
job_request:approve	X	X	X	✓	✓	Slice 2
job_request:reject	X	X	X	✓	✓	Slice 2
job_request:assign-engineer	X	X	X	✓	✓	Slice 2
job_card:read-list	X	✓	✓	✓	✓	GET /job-cards + nav item
job_card:read-detail	X	✓	✓	✓	✓	Slice 2
audit_log:read	X	X	X	X	✓	Phase 7
user:read-list	X	X	X	X	✓	Phase 7

Legend: N = Normal · V = View-Only · E = Lab Engineer · L = Lab In-Charge · S = Super Admin.

Appendix B — Status Badge Colors

Status enum	UI label	Background	Foreground	Hex bg	Hex fg
DRAFT	Draft	slate-100	slate-700	#F1F5F9	#334155
SUBMITTED	Pending	amber-100	amber-700	#FEF3C7	#92400E
ASSIGNED	Approved	violet-100	violet-700	#EDE9FE	#5B21B6
IN_PROGRESS	In Progress	blue-100	blue-700	#DBEAFE	#1E40AF
COMPLETED	Completed	green-100	green-700	#DCFCE7	#166534
VERIFIED_CLOSED	Verified	emerald-100	emerald-700	#D1FAE5	#065F46
REJECTED	Rejected	red-100	red-700	#FEE2E2	#991B1B
REOPENED	Reopened	orange-100	orange-700	#FFEDD5	#9A3412

Status enum	UI label	Background	Foreground	Hex bg	Hex fg
CANCELLED	Cancelled	slate-100	slate-500	#F1F5F9	#64748B

Appendix C — The Phase 6 Decision Log

Every non-trivial decision in Phase 6, captured for future engineers who will want to know why.

#	Decision	Alternative	Why we chose this
D1	Bottom-up layering for backend modules.	Routes-first to "see the shape early".	Bottom-up keeps each layer testable in isolation and matches the import-graph.
D2	Canonical names + repo aliasing.	Real names everywhere.	Schema drift survives in one file (the repo) instead of many.
D3	Single state-machine choke-point.	Inline if/else in services.	One transition table, one audit source, one place to test illegality.
D4	Idempotent additive migrations.	Naive ALTER statements.	Re-runnable; survives partial-failure deploys; rollback is forward-rollback.
D5	Defence in depth (BE + FE schema parity).	FE-only or BE-only validation.	A breach of one layer does not produce a violation.
D6	Audit row in the same transaction as state change.	Audit after commit.	Audit drift becomes impossible by construction.
D7	Atomic-increment code generation.	UUID or SELECT FOR UPDATE.	Human-readable codes; no row locks; no deadlock risk under load.
D8	Thin controllers.	Logic in controllers.	Services are transport-agnostic; controllers are replaceable.
D9	FE schemas mirror BE schemas (duplicated).	Shared package.	Cheap to maintain; avoids cross-build coupling.
D10	URL is source of truth for list state.	useState in component.	Bookmarking, deep linking, refresh all work without extra code.
D11	React Query owns server state.	Redux for everything.	Cache-with-invalidation primitives are exactly what list/mutation flows need.
D12	Stateless presentational components.	Data-fetching components.	Each component has one reason to change.
D13	Two-call create-then-submit.	Single create-with-submitted flag.	Recovery: a failed second call leaves the draft intact.
D14	T&C double-lock (FE button + BE literal).	FE-only.	A clever client cannot bypass the gate.
D15	Snapshot submitter fields, not just FK.	Always JOIN users.	Audit integrity survives user renames and deactivations.
D16	JSON column for accessories, not child table.	Child table.	Accessories are read/written as a group; never queried independently.
D17	rowLevelScope as middleware, not per-service.	Branching in every list service.	One file owns the rule; less drift surface.

#	Decision	Alternative	Why we chose this
D18	FULLTEXT + LIKE fallback for search.	LIKE only.	Sub-millisecond search at 100k rows; LIKE handles short terms.
D19	Disable Export button instead of hiding it.	Hide entirely.	Telegraphs intent; users know the feature is coming.
D20	Disabled identity fields in form Section 4.	Read-only labels (no inputs).	Looks like a form field; conveys "do not touch" without breaking the visual rhythm.
D21	Pagination block "1 2 ... 99 100" design.	Just Prev/Next.	Power users jump pages; window keeps the block short.
D22	30-second list staleTime.	0 (always refresh) or never.	Balances freshness vs server load for typical MIS use.

Appendix D — Files Added & Patched (Reference Count)

Layer	New files	Patched files
Backend — SQL discovery + migrations	5	0
Backend — utilities (audit, codegen)	2	0
Backend — jobRequests module	6	0
Backend — jobCards module	5	0
Backend — lookups module	3	0
Backend — middleware + users + server	0	3
Frontend — API hooks	3	0
Frontend — zod schemas	2	0
Frontend — shared components	3	0
Frontend — Job Requests pages	8	0
Frontend — Job Cards pages	2	0
Frontend — routing + nav + Tailwind	0	3
TOTAL	39	6

Appendix E — Glossary Quick-Reference

Term	Meaning
Acceptance criterion	A numbered claim about system behaviour. Each maps to one smoke step.
Choke-point pattern	One function owns all state changes for an entity. No bypasses allowed.
Covering index	An index that includes every column the query needs. EXPLAIN shows "Using index".
Defence in depth	The same rule enforced at multiple layers, each sufficient on its own.
Doctrine	A binding law of the codebase, written down. Eight doctrines govern Phase 6.
Idempotent migration	Safe to apply multiple times. Achieved via INFORMATION_SCHEMA guards.
Layered module	Six files in dependency order: validators → state-machine → repo → service → controller → routes.
Row-level scope	Middleware-set filter that constrains a list query to rows the caller may see.
Sealed phase	Frozen artifacts; additive amendments only; no destructive changes.
Slice	A vertical, end-to-end shipping unit of a phase.

Term	Meaning
Snapshot column	Denormalised value copied at write-time so future reads survive source changes.

“Doctrine is what you write down so the next engineer does not have to re-derive it.”

★ **End of Phase 6 sealed reference**

Twelve parts. Five doctrines for the backend, three for the frontend. Eight modules and two utilities. Three screens. Twelve acceptance criteria. Twenty-two recorded decisions.

This document is your map for Slice 1 and the rails on which Slice 2 will be built.

Future engineers: amend by addition only. The doctrine is a contract; the appendices are the receipts.